

Background Reference

BLAST and Sequence Similarity

From biological intuition to command-line analysis

Computing for Data-Driven Biology · 5BI00A
Master's Programme in Bioinformatics · Umeå University

Session Aims & ILO Links

- 1 Explain what sequence similarity is and why it is biologically meaningful
- 2 Describe how BLAST works — seeding, extension, scoring, and E-values
- 3 Interpret a BLAST results table including E-values, bit scores, and % identity
- 4 Distinguish the five BLAST programmes and select the appropriate one
- 5 Choose an appropriate BLOSUM matrix for a search — default, distant, or close relatives
- 6 Recognise common interpretation pitfalls — coverage vs. identity, and database-size effects

ILO 5

ILO 1

ILO 7

ILO 5

ILO 5

ILO 7

This is background reading, not a taught session. Read it before the "Three ways to run BLAST" hands-on exercise in Lecture 15, where you will use your TP53 sequence from Lecture 11.

Part 1

Sequence similarity — the biological foundation

Similarity Implies Common Ancestry

Sequences that are similar are likely related by evolution — they are homologous.

Homologous sequences often share biological function because structure is conserved by natural selection.



Orthologues

Genes in different species descended from the same ancestral gene via speciation. Usually retain the same function. This is the basis of functional annotation transfer across species.



Paralogues

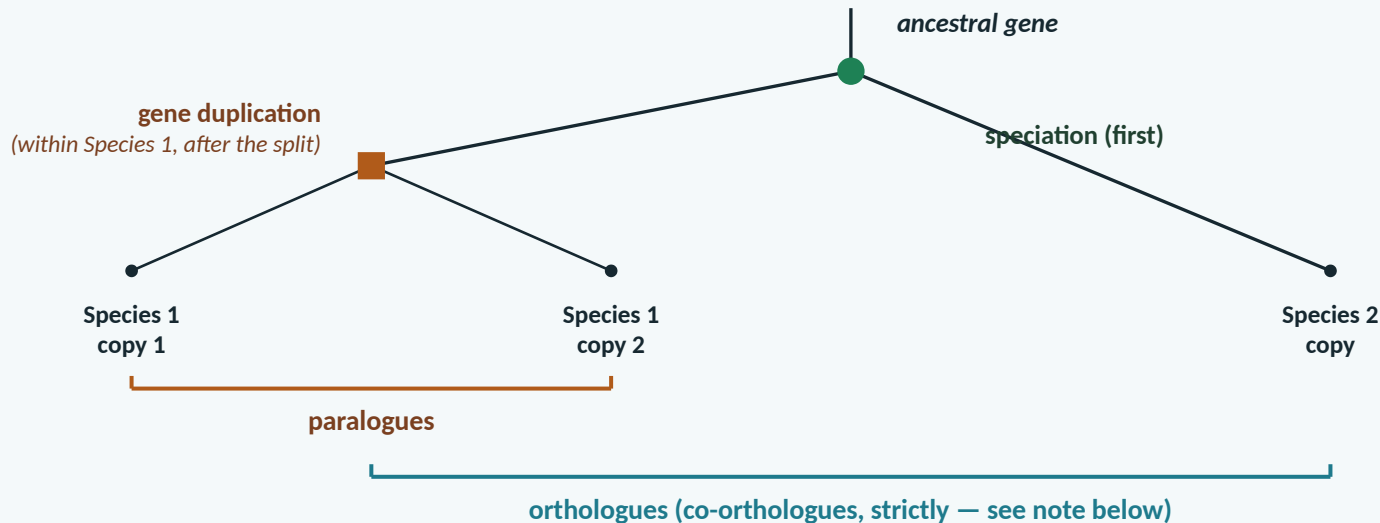
Genes within the same genome related by gene duplication. Often diverge in function — one copy may retain the ancestral role while the other acquires a new one. Be cautious about annotation transfer between paralogues.

Similar sequence $\neq$ identical function.

The top BLAST hit is a hypothesis about function, not a fact. The strength of that hypothesis depends on E-value, % identity, query coverage, and the reliability of the subject sequence's own annotation.

Orthologues and Paralogues on a Gene Tree

The same gene tree tells you whether two sequences are related by speciation or by duplication.



Because Species 1 now has two copies, each is technically a **co-orthologue** of the single Species 2 copy, not a strict one-to-one orthologue — but the practical question for interpreting a BLAST hit is the same either way: duplication, or speciation?

Why Do You Still Get Hits Within the Same Species?

Searching a genome or CDS set against itself is not a null result — it routinely finds real hits. Here is why.

One genome, one chromosome: three orange genes are members of the same recently-duplicated family — a self-BLAST lights all three up together.



Paralogy / gene families

Many genes belong to families created by past duplication. A self-search finds every family member, not just “the” gene.

Shared functional domains

Unrelated gene families can share one conserved domain (e.g. a kinase domain, a zinc finger). The hit reflects the domain, not the whole gene.

Low-complexity / repetitive sequence

Simple repeats, polyA tracts, and transposable-element-derived sequence recur genome-wide and align to themselves everywhere.

Recent tandem duplicates

Very recent duplicates or allelic copies are still near-identical — a real, biologically meaningful hit, just not a cross-species one.

Part 2

How BLAST works — a conceptual overview

Global vs Local Alignment

Two different questions you can ask when comparing two sequences.

Global (Needleman–Wunsch)



Both sequences are forced into one alignment end-to-end, gaps included, across their entire length.

Appropriate when you already expect the two sequences to be homologous over their full length (e.g. two alleles of the same gene).

Local (Smith–Waterman / BLAST)



Only the best-matching region is found and reported — this is the **HSP**.

Flanking dissimilar sequence is ignored entirely, even if the rest of the two sequences share no similarity at all.

BLAST is a LOCAL method — the “L” in BLAST. Exhaustive local alignment (Smith–Waterman) exists and finds the **mathematically best** local alignment — but, like global alignment, it is too slow to run against a database of billions of sequences. BLAST's **heuristic** seed-and-extend approach approximates Smith–Waterman fast enough to be practical at that scale.

The Problem BLAST Solves

You have one query sequence. The database has billions. How do you find all similar sequences quickly?

Naïve approach

Compare every residue of the query to every residue of every sequence in the database. Guaranteed to find the optimal alignment — but computationally impossible for large databases.

BLAST — heuristic approach

Use short-word seeds to rapidly identify candidate matches, then only extend full alignments from those seeds. Fast enough for billions of sequences. Does not guarantee finding the optimal alignment — but finds the biologically important ones.

Heuristic = fast and good enough. Not exhaustive. Understanding this limitation is part of interpreting BLAST results responsibly.

Seed and Extend: The Two-Stage Strategy

1 Seeding

Break the query into short overlapping words (k-mers). Find exact or near-exact matches to these words in the pre-indexed database. Fast — uses a lookup table. Each match is a seed.

2 Extension

Extend each seed in both directions, one residue at a time, as long as the alignment score keeps improving. Stop when the score drops too far below the best seen. The result is a High-Scoring Pair (HSP).

3 Reporting

Rank all HSPs by score. Calculate E-values. Report those above the significance threshold. Multiple HSPs for the same subject sequence may be merged.

Word size k: typically 11 nt for blastn, 3 aa for blastp. Smaller k = more sensitive but slower. Larger k = faster but may miss distant homologues.

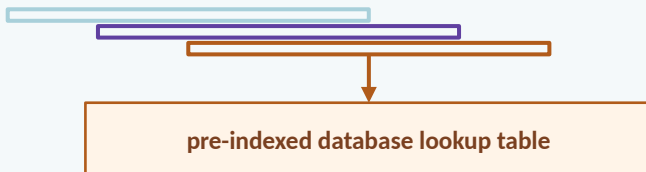
Word Size and Seeding, Visually

How BLAST finds candidate matches before it ever computes a full alignment. Shown here with a short word size ($k=4$) for clarity — blastn actually uses $k=11$.

Query

G	A	T	T	A	C	A	G	C	T	T	G	A	C	C
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

word size $k = 4$ — window slides one position at a time; the highlighted window finds a match



Subject

C	A	T	T	A	C	A	G	C	T	T	G	T	C	C
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



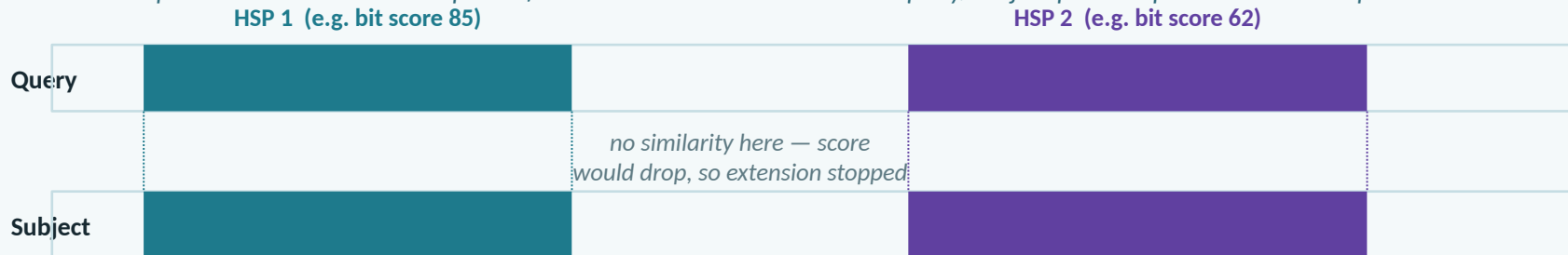
extend — score improving

stop — score drops

Word size = seed length. Smaller $k \rightarrow$ more candidate seeds found $\rightarrow$ **more sensitive but slower**. Larger $k \rightarrow$ fewer seeds $\rightarrow$ **faster but may miss short or weak homologies**. Real defaults: $k=11$ (blastn), $k=3$ (blastp).

Why One Alignment Becomes Multiple HSPs

Extension stops as soon as the score drops too far below the best seen — so one query/subject pair can produce several separate HSPs.



A single BLAST search can report several HSPs for the same query/subject pair — they are ranked and scored **separately**, not merged into one alignment.

Common real causes: a spliced cDNA/CDS query aligned against unspliced genomic DNA (each exon becomes its own HSP, separated by an intron)

• domain shuffling or large indels between two otherwise-related proteins • a large insertion, inversion, or rearrangement between two genomes.

Where Do BLOSUM Scores Come From?

Not a guess — measured from thousands of real, conserved protein alignments.

1. Start from a “block”

A short, ungapped, reliably-aligned region from many related proteins (the BLOCKS database):

W	sequence 1 — same column across the block
W	sequence 2 — same column across the block
W	sequence 3 — same column across the block
F	sequence 4 — same column across the block
Y	sequence 5 — same column across the block

2. Count how often each amino acid pairs with each other in this column, across thousands of blocks, and compare to how often that pairing would happen by chance alone.

3. Score = log-odds

$$\text{score}(a,b) = \log_2(\text{observed}(a,b) / \text{expected}(a,b))$$

expected(a,b) assumes random pairing based on each amino acid's own background frequency.

Trp–Trp (W–W) → BLOSUM62 = +11

Rare amino acid, but conserved almost perfectly when it occurs — far more than chance predicts.

Trp–Asp (W–D) → BLOSUM62 = -4

Bulky aromatic vs. small acidic — essentially never substitutes for it in real conserved blocks.

Values verified against the published NCBI BLOSUM62 matrix (<ftp.ncbi.nlm.nih.gov/blast/matrices/BLOSUM62>).

Scoring: Substitution Matrices

Protein BLAST scores are not simply +1 for match, -1 for mismatch. Each amino acid pair has its own score based on how often they are observed to substitute for one another in real, related proteins.

BLOSUM matrices

Derived from observed substitutions in conserved blocks of related proteins.

BLOSUM80

Closely related sequences (< 80% identity used)

BLOSUM62

Moderately diverged — DEFAULT for most searches

BLOSUM45

Distantly related sequences — more sensitive

What the number means

BLOSUM62 is derived from sequences sharing $\leq 62\%$ identity. The lower the number, the more diverged the reference sequences — and the better the matrix is at detecting distant relationships.

Conservative substitutions (e.g. Leu→Ile, similar chemistry) score positively. Radical substitutions score negatively. This is why protein BLAST is more sensitive than nucleotide BLAST for distant homologues — amino acid chemistry is informative in a way that nucleotide identity alone is not.

In practice: use BLOSUM62 for most searches. The Comparative Genomics course covers matrix selection in depth.

The E-value: The Most Important Number in Your Results

E-value = how many times would you expect to find a match this good (or better) by chance in a database of this size?

0.0 ($\approx 1e-180+$)

Certain

Astronomically unlikely to be coincidental. Essentially certain homologue.

$1e-50$

Very strong

Very strong hit. Highly conserved sequence. Almost certainly true homologue.

$1e-10$

Strong

Strong hit. Likely homologue. Worth investigating.

$1e-5$

Moderate

Moderate. Generally accepted threshold for annotation transfer. Check carefully.

0.01

Weak

Weak hit. Treat with scepticism. May be real or may be noise.

≥ 1.0

Noise

Expected to occur by chance. Almost certainly not biologically meaningful.

⚠ E-value depends on database size. The same alignment gives a lower E-value against a small database than a large one. Always report which database you searched.

Why Protein Search Beats DNA

The genetic code is redundant — not every DNA change alters the protein.

Synonymous (silent) change

C	T	T	→ Leu (L)
C	T	C	→ Leu (L)
C	T	A	→ Leu (L)
C	T	G	→ Leu (L)

All four 3rd-position variants still encode Leucine. **The protein is unchanged.**

Non-synonymous change

C	T	T	→ Leu (L)
---	---	---	-----------

1st position changes:

G	T	T	→ Val (V)
---	---	---	-----------

A single base change swaps Leucine for Valine. **The protein changes — selection can now act on it.**

Synonymous substitutions accumulate **neutrally** over time — nothing selects against them. Non-synonymous substitutions are **constrained by purifying selection** on protein function.

Result: DNA sequence identity decays faster than protein identity as lineages diverge. Beyond a certain distance, blastn can no longer detect a real homology that blastp or tblastx still can.

When Does Programme Choice Matter?

Evolutionary distance between query and database determines how much sensitivity you need.

increasing evolutionary distance →

Same genome / same species	Close relatives (same genus)	Different families / orders	Cross-kingdom, ancient split
blastn is fine	blastn usually still works	blastp or tblastx recommended	blastp / tblastx required
Nucleotide identity still very high (>95–99%).	Identity often high enough (~85–95%) for reliable local alignment.	blastn sensitivity drops sharply as DNA identity decays.	blastn frequently fails outright — protein signal may be all that remains.

The mechanism: protein sequence is more conserved than the underlying DNA (see the previous slide) — so protein-level search stays sensitive for longer.

The Five BLAST Programmes

blastn

Query: Nucleotide

DB: Nucleotide

Find similar DNA/RNA sequences. Identify a species from a sequenced fragment (e.g. 16S rRNA, COI barcode). Fast but less sensitive for distant homologues.

blastp

Query: Protein

DB: Protein

Find similar protein sequences. Most sensitive for distant evolutionary relationships. Default choice for functional annotation.

blastxQuery: Nucleotide
(translated)

DB: Protein

Find protein matches for an unannotated nucleotide sequence. Translates in all 6 reading frames. Useful when you have a new genomic sequence but no gene model.

tblastn

Query: Protein

DB: Nucleotide
(translated)

Find unannotated genomic regions that encode a protein of interest. Useful for identifying new gene models in an unannotated genome.

tblastxQuery: Nucleotide
(translated)DB: Nucleotide
(translated)

Most sensitive nucleotide-to-nucleotide search (via protein translation). Slow. Used for finding distant nucleotide homologues where blastn fails.

Rule of thumb: protein-level comparisons (blastp, blastx) are more sensitive than blastn for detecting distant homologues.

Interpreting the BLAST Results Table

Description

Name of the matching database sequence (subject)

E value

Expect value — how likely this match is by chance. Your primary significance indicator.

Per. ident %

Percentage of identical residues in the aligned region. Always check alongside query cover.

Query cover

Fraction of your query sequence covered by the alignment. 10% coverage at 90% identity $\neq$ 90% coverage at 90% identity.

Bit score

Database-size-independent alignment quality score. Use for comparing across different searches.

Accession

Database accession number. Use to fetch the full sequence or annotation.

 **High identity over a short region $\neq$ same overall function. Always check BOTH percentage identity AND query coverage together.**

A Hit List Is Not the Answer — Build a Tree

E-value and % identity rank by similarity, not by evolutionary relationship. (Illustrative example.)

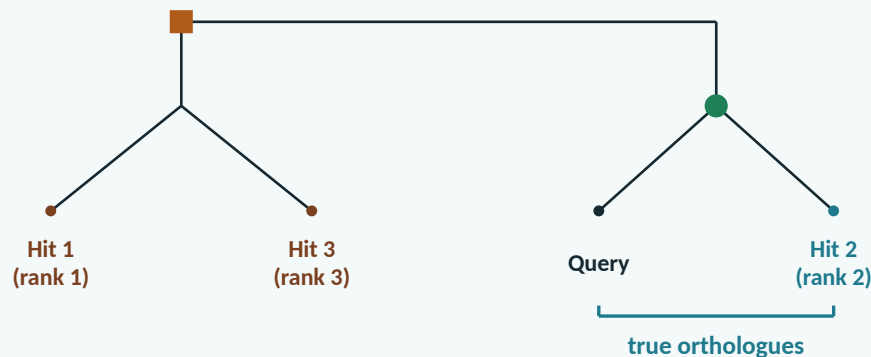
Ranked BLAST hits

Rank	Subject	% id	E-value
1	Species B, copy 2	78%	2e-90
2	Species C ortholog	71%	4e-85
3	Species B, copy 1	69%	1e-80

Rank 1 has the best E-value and % identity — but it is a **paralog** from a lineage-specific duplication in Species B, not the **true orthologue** (rank 2).

The tree resolves it

Hit 1 and Hit 3 branch off **before** the speciation event that separates Query from Hit 2 — they are paralogues, however similar their raw scores looked.



Rule of thumb: whenever a gene has known paralogues, build a tree (or at least align against well-characterised reference sequences) before trusting the top hit as "the" orthologue.

What You Should Know From This Material



Explain what homology means and distinguish orthologues from paralogues



Describe the seed-and-extend strategy of BLAST at a conceptual level



Explain what an E-value is and interpret one correctly, including the effect of database size



Distinguish bit score from E-value and explain when bit score is more appropriate



Select the correct BLAST programme for a given query and database type



Interpret a BLAST results table: coverage, % identity, E-value, bit score



Choose an appropriate substitution matrix (BLOSUM62 default; BLOSUM45/80 for distant/close)



Recognise common pitfalls: query coverage vs. percentage identity, and why E-values depend on database size



Distinguish orthologues from paralogues in database hits

The hands-on version of this exercise (with PlantGenIE) is taught live in Lecture 15 — see the Lecture 15 handout.